

Analog Signal - XADC - Basys 3 SPI - Arduino Practice

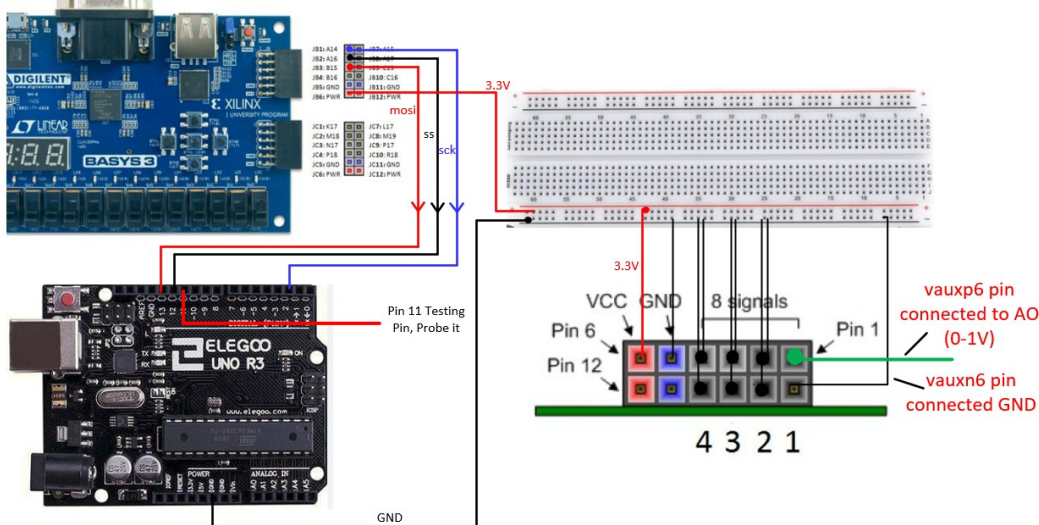
This tutorial teaches how to integrate the XADC IP block into your FPGA design and access the digital output and transmit them to an external MCU through

The analog input (AO) must be 0-1V. If you are taking CE433 and you have the Easy Pulse Sensor module, you should use that output (adjust it to 0-1V) as

The hardware connections follow. Note that AO is connected to vauxp6; and vauxn6 is shorted to GND. The vauxp/n pins are at the JXDC pins (bottom left). All other ADC channels are grounded to avoid noises.

GND from Arduino is used to connect to the common GND of the entire circuit however the VCC (5V) of Arduino shouldn't be used on Basys3 (3.3V device). The SPI module in the FPGA will send sck, ss, and mosi to Arduino so it's a one-directional communication which won't hurt the FPGA board.

Basys3: Pmod Pin-Out Diagram



JB1 - 3 are used as sck, ss, and mosi respectively. Follow the procedure in the XADC tutorial to add an XADC IP to this design.

```

154 #Pmod Header JB
155 #Sch name = JB1
156 set_property PACKAGE_PIN A14 [get_ports {sck}]
157 set_property IOSTANDARD LVCMOS33 [get_ports {sck}]
158 #Sch name = JB2
159 set_property PACKAGE_PIN A16 [get_ports {ss}]
160 set_property IOSTANDARD LVCMOS33 [get_ports {ss}]
161 #Sch name = JB3
162 set_property PACKAGE_PIN B15 [get_ports {mosi}]
163 set_property IOSTANDARD LVCMOS33 [get_ports {mosi}]
164 ##Sch name = JB4
165 #set_property PACKAGE_PIN B16 [get_ports {JB[3]}]

```

The FPGA board is now used as the leader transmitter, it outputs voltages to the 5V logic Arduino so NO level shifter is needed in this circuit.

On the Arduino board, Pin 13 is mosi, Pin 12 is ss, and Pin 2 is sck.

Pin 11 is used as a testing pin in your C code. This is a very handy technique to debug your circuit. If you are taking my MCU class in the fall, you will use it quite often.

This is how Pin 11 creates a pulse to indicate that this piece of code is executed.

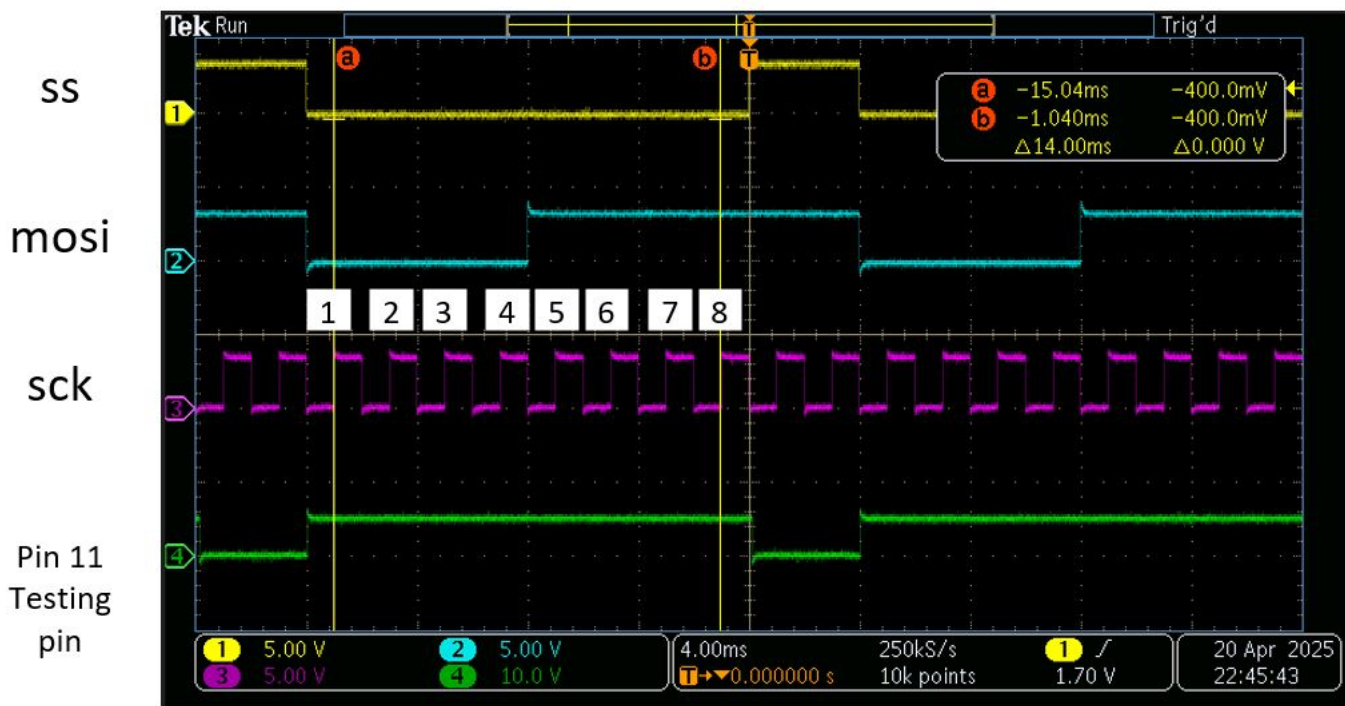
```

void loop() {
  ssPin_state = digitalRead(ssPin);
  if (ssPin_state != lastssPin_state) {
    if (ssPin_state == LOW) {
      data=0; // this fixed a lot of things
      digitalWrite(11,HIGH);
      for(i=0;i<7;i++){
        while (digitalRead(sckPin)==0);
        data|=digitalRead(mosiPin);
        data<<=1;
        while(digitalRead(sckPin)==1);
      }
      while (digitalRead(sckPin)==0);
      data|=digitalRead(mosiPin);
      while(digitalRead(sckPin)==1);
      Serial.println(data);
      digitalWrite(11,LOW);
    }
  }
  lastssPin_state=ssPin_state;
}

```

The [complete Arduino code](#). Note that the two first two lines of code right below the for loop seem to be unnecessary, test it and let me know.

Here is the observation. The mosi line comes from the FPGA. I wrote a constant value 8'b00001111 to the mosi output from the FPGA for debugging purp



In the [SPI_leader_transmitter.v code](#), at the negedge of the sck, ss is pulled down to 0 at the same time to enable the follower device. From the observ the left handside of the label '1'. The falling edge of sck, mosi, and ss almost happened at the same time. Waiting for a half sck cycle to sample the data at the rising edge of the sck clock will be perfect (the yellow cursor labeled '1' or 'a' samples a '0' from mosi). The 'a' cursor samples a 0 from mosi, and MSB of the byte '8'b00001111' sent through the FPGA.

After sck rising edge No. 1, you can count to 8 sampled bits by Arduino and the 8th rising edge is right before the 'ss' enable line got pulled up to disable transmission. This signals the completion of transmission of 1 entire byte.

The Testing pin is pulled up and down at the correct time points which indicates that the Arduino code is executed as expected.

Observing the output from the FPGA should be the first step of this project since we need to know that the SPI leader transmitter module is functionin on.

I connected the entire circuit to Arduino and observed the output to the Serial monitor. I received '15' constantly there so this is great news.(8'b000001

Here is how I output a constant data to mosi.

```

36     .channel_out(),
37     .drdy_out(ready)
38 );
39 SPI_leader_transmitter(.clk(CLK100MHZ),.data(data_latch),.sck(sck),.send(send),.ss(ss),.mosi(mosi),.busy(SPI_busy));
40 always @(posedge (CLK100MHZ)) begin
41     if(ready==1)begin
42         led<=data[15:8];
43         // data_latch<=data[15:8];//output ADC data
44         data_latch<=8'b00001111;//output a constant for debugging
45     end
46     if (SPI_busy==0)begin
47         send<=1;
48     end
49     else send<=0;
50 end
51 endmodule

```

Code for the [top module main.v](#)

The [constraint file](#).

Switch it back to the commented one above will deliver ADC outputs to mosi.

Final deliverables and rubric

Task 1:

XADC outputs constant data (8'b00001111) to an oscilloscope. If you oscilloscope only has two channels, please display sck and mosi. (25 points)

Task 2:

Use the Easy Pulse Sensor as AO. Signal outputs to an Arduino IDE in the Serial plotter. Show you verilog and Arduino code. A video demo is required for points)